

Software Engineering Experience

Describe a skill or knowledge you acquired recently that has been impactful for you. Why did you make this investment? What has the outcome been?

Over the past year, I invested seriously in learning about multi-agent systems and agentic AI frameworks like LangGraph and LangChain. My motivation was practical: I kept running into the limits of single-pass LLM calls when building AI-assisted features. I wanted to understand how you could decompose a complex and multi-step task, something like automating a GitHub workflow into a graph of specialized agents that could reason, plan, and self-correct.

So I built AutoPR. It takes a GitHub issue, analyses the repository structure, plans a code and test patch, repairs the patch if it fails validation, and opens a pull request and all autonomously. Building it forced me to think carefully about state management between agents, failure recovery, and how to design agent boundaries so that each one has a well-defined contract. It also sharpened my Python considerably. The skills have already carried over into my professional work at Sacesta Technologies, where I applied similar pipeline thinking to building a real-time AI voice system.

What new skill would you like to learn? Why do you think this is important or timely or interesting? Why do you think you will be good at it?

I want to go deep on Linux systems programming and kernel internals and understand how the kernel manages processes, memory, scheduling, and how that interacts with the user-space software running on top of it. I have been using Ubuntu daily for three years, but I am aware that my knowledge sits mostly at the application layer. I want to understand what is happening underneath.

I think this is the right moment for me to make this investment. The industry is shifting toward more efficient systems like Ubuntu Core on IoT devices and containerized workloads on Kubernetes. Understanding the kernel and runtime layer is what separates engineers who can reason about performance and correctness at a deep level from those who can only observe symptoms at the surface. Canonical is exactly the kind of environment where this knowledge is not a nice-to-have but it is central to the work.

I think I will be good at it because I have always been drawn to understanding systems from the ground up. My CGPA reflects consistent academic performance in courses like Operating Systems, Networking, and Databases. I have 900+ LeetCode problems solved, which has trained me to think algorithmically under constraints. And projects like AI Frontdesk where I had to reason about latency, WebRTC transport, and real-time audio pipelines show that I am comfortable operating good enough even without formal systems training.

What kinds of software projects have you worked on before? Which operating systems, development environments, languages, databases?

My projects span backend systems, real-time communication, AI pipelines, and developer tooling. Professionally, I have built the backend for a global eSIM commerce platform serving 190+ countries at Sacesta Technologies, engineered real-time WebSocket-based event systems supporting 300-500 weekly active users, and implemented a caching and reporting infrastructure at Medkart Pharmacy that cut search latency by 60% and saved 5+ operational hours per week across 50+ store managers.

On the personal side, I have built a real-time collaborative whiteboard (CollabCanvas), an AI voice assistant with supervisor escalation (AI Frontdesk), a multi-agent GitHub automation tool (AutoPR), and a full-stack Chrome extension with a PostgreSQL backend (LinkedIn Saved Posts Explorer).

I work primarily on Ubuntu (daily driver for three years). My most-used languages are JavaScript and TypeScript (Node.js, Express, React), Python, Go, SQL, and Bash. I have also worked with C++ and PHP. For databases, I use PostgreSQL most heavily, along with MySQL, MongoDB, Redis, and QdrantDB for vector search. On the infrastructure side, I have worked with Docker, AWS (Lambda, SQS, EC2, S3, RDS), Prometheus, and Grafana.

Would you describe yourself as a high quality coder? Why? When you are looking at code, how do you decide if it is high quality?

Yes, I would but not because I think I have nothing left to learn, but because quality is something I care about deliberately. When I write code, I am thinking about the person who has to read it six months later, whether that is a teammate or a future version of myself. That means meaningful variable names, functions that do one thing, and logic that is explicit rather than clever. I write modular code as well as I follow best practices such as following Solid principles and design patterns wherever required.

When I am evaluating whether code is high quality, I look at a few things. First is readability where I check can I understand what the code is doing without running it or having the author explain it? Second is testability where I check are the units small enough to be tested in isolation, and are the side effects explicit? Third is correctness at the edges where I check: does the code handle error cases and unexpected inputs, or does it only work on the happy path? Finally, I look at whether the abstractions are honest, whether the names and structure reflect what the code actually does, or whether there is a mismatch that will confuse future readers.

The 70% latency improvement I achieved on image preview generation at Sacesta, and the 60% search latency reduction at Medkart, came from reading existing code carefully enough to understand not just what it was doing but why it was slow and that kind of reading requires the code to have been written well enough to reason about. Good code makes optimization possible while bad code makes it tough.

Outline your thoughts on open source software development. What is important to get right when you are working on open source projects?

Open source is fundamentally a collaborative contract. You are writing code that others will depend on, contribute to, and build their own software around. That changes what quality means. In a private codebase you can tolerate some implicit knowledge shared among a small team. In open source the code has to be legible and reliable to someone who has never spoken to you.

The things I think are most important to get right: clear contribution guidelines, consistent code style enforced by tooling rather than convention, thorough documentation that explains the why and not just the what, meaningful commit messages, and a review culture that is rigorous but welcoming to newcomers. It is important to test each and every contribution thoroughly before being merged.

My direct open source experience is still developing. I have worked with open source tools like LangGraph, LangChain, Socket.io, LiveKit and I have studied how projects like these handle contributions, issue triage, and API design decisions. My AutoPR and CollabCanvas projects are publicly available on GitHub.

Please write about your familiarity with Canonical's primary languages, prioritizing the ones you know best. How extensive is your experience? How do you test applications in these languages?

JavaScript and TypeScript are my strongest languages, with around two years of professional experience. I have built production REST APIs in Node.js/Express, real-time systems with Socket.io, and frontend applications in React. I test JavaScript applications using Jest for unit and integration tests, and I have used Postman for API endpoint testing. I understand the event loop well enough to reason about performance bottlenecks and async patterns.

Python is my second strongest language. I use it for backend services (FastAPI), AI pipelines (LangGraph, LangChain), and scripting. I have built production-grade async Python services, and I test with pytest, using fixtures and mocking for external dependencies like LLM APIs and WebRTC sessions.

Go is a language I have used for personal projects and am actively deepening. I am drawn to its simplicity, strong concurrency primitives, and the way it forces you to handle errors explicitly.

Bash I use daily for scripting, automation, and working with the Linux environment — cron jobs, deployment scripts, process management. I am comfortable writing non-trivial shell scripts.

C++ I studied formally and used in solving DSA questions on leetcode and competitive programming and coursework. I understand memory management, pointers, and the language's performance model, though I have not used it in a production setting yet.

Rust is on my immediate learning roadmap.

How comprehensive would you say your knowledge of networking is? How have you gained this knowledge?

My networking knowledge is solid at the application and transport layers, and I am comfortable reasoning about distributed systems concerns like latency, retries, and backpressure. I have worked with WebSockets for real-time bidirectional communication, WebRTC for peer-to-peer audio transport, HTTP/REST and GraphQL for service communication, and message queues like AWS SQS for async workloads. I have configured networking in Docker and AWS environments, worked with VPCs, security groups, and load balancers as part of personal learning.

At the lower levels I know about TCP/IP internals, the kernel's networking stack, socket programming as I have academic grounding from my Networking course at Ahmedabad University, and I understand the fundamentals well. But I am honest that I have not done production work at the kernel networking level, and that is an area I am actively looking to develop.

How have you used relational data and SQL databases in your work? How much data model design work have you done?

PostgreSQL is my primary database. I have designed schemas from scratch for multiple projects — the LinkedIn Saved Posts Explorer, the cloud storage platform, and the Medkart reporting system. I think carefully about normalization, appropriate use of indexes, and foreign key constraints. I have used date-range filters, multi-table joins, and aggregation queries for reporting across 50+ stores.

The most unexpected thing I encountered in structured data model design is how often the naive schema that feels natural early on becomes a bottleneck at scale and not because the design is wrong, but because the access patterns shift as the product grows. At Medkart, what started as a simple product search query became a multi-table join once multi-store aggregation was added. That taught me to think about access patterns early and to be deliberate about denormalization when read performance matters more than write simplicity.

I have also worked with MySQL and have used AWS RDS for managed relational databases in production.

Which NoSQL or unstructured data stores have you used? How do you reason about choosing the right data store for a project?

I have used MongoDB, Redis, and QdrantDB in production. MongoDB I have used for flexible document storage where the schema is evolving quickly or where the data is naturally hierarchical. Redis I use for caching and session state and it was central to the 60% latency improvement I achieved at Medkart. QdrantDB I used for vector similarity search in an AI application, which required understanding approximate nearest-neighbour indexing.

My reasoning for choosing a data store starts with access patterns and consistency requirements. If the data is highly relational and consistency is critical such as financial transactions, inventory then I reach for PostgreSQL. If I need sub-millisecond reads for hot data then I use Redis. If the schema is genuinely unpredictable or documents are the natural unit then I prefer MongoDB. Vector databases like Qdrant are for semantic search use cases where you need to query by embedding similarity rather than exact match.

I would not rule out any store I have used, but I would be cautious about reaching for MongoDB when the data is actually relational as it is a common mistake to pick a document store for flexibility and then spend months working around the lack of joins.

How comprehensive would you say your knowledge of Linux is, from the kernel up?

I have been running Ubuntu as my daily development environment for three years. I am comfortable with the user-space layer like package management with apt and snap, shell scripting in Bash, systemd services, cron jobs, file system permissions, process management, and networking configuration. I have set up Docker environments, written deployment scripts, and debugged production issues on Linux servers.

Below that level the kernel, device drivers, the scheduler, memory management, the networking stack at the socket layer I have academic grounding from my Operating Systems course, and I understand the concepts. But I am honest that my hands-on experience does not yet go to that depth. This is exactly the gap I am most motivated to close, and working at Canonical where the Linux distribution is the product is the environment where I want to be in to develop that knowledge seriously.

Describe any experience with low-level embedded systems engineering.

I do not have direct production experience with embedded systems. My systems work has been at the Linux application and service layer. Hence I am interested in the embedded and IoT space and Ubuntu Core for constrained devices is one of the Canonical products I find most compelling and I see it as an area I would grow into as I deepen my kernel and systems knowledge.

Personal Project: AutoPR

Briefly describe this personal project. What were you hoping to achieve?

AutoPR is a multi-agent system built in Python using LangGraph that automates the full journey from a GitHub issue to a merged pull request. Given a repository and an issue, it analyses the codebase, plans a fix, generates code and test patches, validates them, repairs failures, and opens a PR with a structured description. My goal was to build something genuinely useful for developer productivity which is not a demo, but a system that could handle real repositories and real issues.

What inspired you to write this piece of software?

I was frustrated by how much repetitive, predictable work surrounds software development - triaging issues, locating the right file to change, writing boilerplate tests for obvious cases. I had been working with LangGraph for AI pipeline projects and wondered whether a multi-agent architecture could handle the coordination complexity of a real software workflow, not just a chat interaction. I also wanted to understand the practical limits of LLM-based code generation when you give it structured feedback loops.

What language did you choose, and why?

Python. LangGraph and LangChain have their best support in Python, and the ecosystem around LLM tooling is most mature there. Python also made it easy to interface with the GitHub API and to write the agent orchestration logic in a readable way. For a project centred on agent coordination rather than raw performance, Python was the right tool.

Which part did you find most difficult? What did you learn?

The hardest part was designing the patch repair loop. When the generated code patch fails validation like syntax errors, failing tests and the system needs to understand why and produce a corrected patch without simply regenerating from scratch. I had to think carefully about how to pass structured error feedback back to the agent, how many repair attempts to allow before escalating to a human, and how to avoid infinite loops where the agent makes the same mistake repeatedly.

What I learned is that reliability in agentic systems comes from the quality of your failure feedback, not just the quality of your initial generation. An agent that can see exactly what failed and why will outperform one that only knows it failed.

How did your code perform?

For well-scoped issues on smaller repositories, the system performed well producing correct patches and valid PRs without human intervention. On larger codebases or issues requiring cross-file changes, the quality of the repository analysis step became the bottleneck. The system is more reliable when the issue description is specific and the scope is narrow, which is itself an interesting finding about the limits of context windows in code understanding tasks.

Was there any part you felt a need to get absolutely right?

The agent boundary design. I spent significant time deciding what each agent's responsibility was, what it received as input, and what it was expected to produce. Getting this wrong early would have meant the system was either too tightly coupled — where changing one agent broke others — or too loosely coupled — where the agents could not share enough context to make good decisions. I drew the boundaries around discrete, verifiable outcomes: analysis produces a repository map, planning produces a structured diff plan, generation produces a patch, validation produces a pass/fail with reasons. Each boundary is also a natural point to inspect or override.

Is there anything you would do very differently now?

I would invest more in the repository understanding step earlier. I underestimated how much the quality of the initial code analysis shapes everything downstream — if the agent has a shallow understanding of the repository structure, the plan it generates will be shallow too. I would also add proper observability from the start like tracing each agent invocation with structured logs rather than adding it retrospectively when debugging became difficult.

Education

How did you rank in your final year of high school in mathematics?

Mathematics was consistently one of my strongest subjects throughout school. I sat the JEE (Joint Entrance Examination) — India's national engineering entrance exam, which is among the most competitive standardised tests in the world — and scored in the 96.77th percentile (98.99 percentile in maths), placing me in roughly the top 3% of over a million candidates nationally. The mathematics component of JEE is known for requiring deep problem-solving ability and speed, not just procedural knowledge. That result is the clearest evidence I have of my mathematical ability relative to a large, competitive cohort.

How did you rank in your final year of high school in your home language?

My home language is Gujarati, with Hindi and English also used through school. English became the language I was most academically active in by the time I reached university, and I would describe myself as a strong writer in English and my written communication has been a consistent strength in academic and professional contexts. My high school English performance was solid, though the JEE result is a more meaningful and standardised signal of my overall academic ranking.

Please state your high school graduation results and explain the grading system.

I completed my Class 12 under the Gujarat Secondary and Higher Secondary Education Board (GSEB) and scored 79%. The GSEB system is percentage-based, with 100% as the maximum. 79% represents a pass with good standing in a board known for moderate difficulty. Separately, I sat the JEE Main which is the national engineering entrance examination and scored in the 96.77th percentile. JEE is the more meaningful academic benchmark: over a million students appear annually, and a score in the 96th-97th percentile places a candidate among the top engineering aspirants in India.

Can you make a case that you are in the top 5% in your academic year?

Yes. The most concrete evidence is my JEE Main percentile of 96.77, which by definition places me in approximately the top 3.23% of all candidates nationally in that sitting with over a million students. JEE is a standardised test taken by the most motivated engineering candidates across India, so it represents a genuinely competitive cohort. Being in the top 3% of that group is a meaningful signal.

At university, I maintained a CGPA of 3.33/4.00 at Ahmedabad University. I also ranked in the top 0.25% of over 400,000 participants in the Flipkart GRiD 5.0 national competition - a ranking that required sustained performance across multiple rounds of technical problems. Winning two hackathons during college, including the Descope sponsor track at DeveloperWeek CloudX 2023, and solving 800+ LeetCode problems add further evidence of sustained effort and problem-solving ability above the average.

What sort of high school student were you? Outside of class, what were your interests and hobbies?

I liked to play sports as I found it relaxing and I enjoy playing any sort of sport. I have hobbies such as reading self-help books, keeping basic knowledge of the latest technology trends, and playing musical instruments like flute and tabla.

Which university and degree did you choose? What other universities did you consider?

I chose the B.Tech in Computer Science and Engineering at the School of Engineering and Applied Science, Ahmedabad University, graduating in May 2024. What drew me to AU specifically was its multidisciplinary approach to engineering education. Rather than a purely technical curriculum, AU emphasises breadth such as exposure to design thinking, social sciences, and cross-disciplinary collaboration alongside the core CS fundamentals. I felt that kind of environment would make me a more well-rounded engineer, and I think it did.

I considered other institutions, but AU's model combined with the quality of its faculty and its relatively small class sizes, which meant more direct access to professors made it my preferred choice for the kind of education I wanted.

Overall, what was your degree result and how did it reflect on your ability?

I graduated with a CGPA of 3.33/4.00. The grading system at Ahmedabad University uses a 4.0 scale, where 4.0 is the maximum. My result reflects solid, consistent performance across a rigorous curriculum that included Data Structures and Algorithms, Operating Systems, Machine Learning, Networking, and Databases.

I think the CGPA fairly represents my academic ability, though I would note that my most meaningful outputs during university were not coursework-based — they were the projects, hackathon wins, and competitive programming achievements that I drove independently. The top 0.25% ranking at Flipkart GRiD 5.0, for instance, required a calibre of algorithmic problem-solving that coursework grades do not fully capture.

During all of your education, can you describe any achievements that were truly exceptional?

The achievement I am most proud of is ranking in the top 0.25% at Flipkart GRiD 5.0 which was a national-level engineering competition run by Flipkart with over 400,000 participants. The competition spanned multiple rounds covering software engineering, systems thinking, and algorithmic problem-solving. Placing in the top 0.25% of that field is the result I would point to as genuinely exceptional.

Beyond that: winning the Desclope sponsor track at DeveloperWeek CloudX 2023 Hackathon, winning the Website Making Challenge 3.0 at Ahmedabad University, and scoring in the 96.77th percentile of JEE Main. Collectively, these represent consistent high performance across academic, competitive, and creative technical challenges.

What leadership roles did you take on during your education?

I was an active member of the Programming Club at Ahmedabad University where I took on a substantive role in organising hackathons and coding competitions for students across the university. Beyond participating I set problems for these competitions which requires a deeper understanding than simply solving them. Designing a good competitive programming problem means understanding what concept or skill you want to test, what the edge cases are, what the difficulty gradient should look like, and how to write a clear problem statement.

Running these events also involved coordinating logistics, recruiting participants, and building a culture of technical challenge within the club. It was the kind of initiative I drove because I believed it was valuable not because it was required.

Did you participate in any social or community initiatives during your education?

Through the Programming Club, the competitions and hackathons I helped organise served a genuine community function giving students who were interested in software development a structured, competitive environment to test themselves and learn. Many participants were encountering competitive programming for the first time. I think of that as a meaningful community contribution within the university, even if it was not a formal social programme.

Context

What gets you most excited about this role?

The chance to work on software that matters at scale, to be close to the Linux ecosystem I want to understand deeply, and to do it alongside people who are genuinely expert in the domains I am most eager to learn. I am at a stage in my career where the environment I am in will shape the engineer I become. Canonical is the environment I want.

Practically, I am excited about the possibility of contributing to Ubuntu Desktop, Canonical Kubernetes, and the Snap ecosystem are the products I already use and care about. There is something motivating about fixing a bug or shipping a feature in software that you know is running on millions of machines around the world.

What would you most want to change about Canonical?

It is difficult to give a well-grounded answer to this without having worked at Canonical.

What gives you the most satisfaction at work?

Solving a problem that felt genuinely hard before I started. Not hard in the sense of complicated or time-consuming, but hard in the sense that the path forward was not obvious where I had to understand something deeply before I could make progress. The 70% improvement in image preview load time at Sacesta is an example: it required understanding the full processing pipeline, identifying where time was actually being spent, and then choosing an intervention that was both effective and maintainable. That kind of problem, where the solution requires real understanding, is what I find most satisfying.

I also get a lot of satisfaction from seeing other people benefit directly from something I built and a feature that removes friction, a performance improvement that makes a workflow noticeably faster, a tool that a colleague uses every day. The feedback loop is short and concrete, which I find motivating.

Which Canonical products would you most like to work on?

Ubuntu Desktop and Canonical Kubernetes are the two I am most excited about. Ubuntu Desktop because it is the product closest to developers as improving it has a direct multiplier effect on the productivity of millions of people who use it as their daily environment. Canonical Kubernetes because the container orchestration space is where a huge amount of infrastructure work is happening, and I have direct experience building and operating containerised systems with Docker and AWS.

Snaps also interest me greatly. The idea of a universal sandboxed self-updating application format that works across Linux distributions is an elegant solution to a genuinely hard packaging problem. As I deepen my systems knowledge I expect Snap internals would become an increasingly interesting area to contribute to.

Why do you most want to work for Canonical?

Honestly, it is the combination of the problem domain and the people. I have been using Ubuntu for three years. The software Canonical builds is software I interact with every day. Working on something that close to the infrastructure I care about — and doing it alongside engineers from diverse backgrounds across the world — is exactly the kind of environment I want to be in early in my career.

I also think Canonical's remote-first, globally distributed model is genuinely differentiated. I learn best when I am working with people whose mental models and approaches are different from mine. A team where someone in Berlin, someone in Lagos, and someone in Bangalore are all working on the same problem, bringing different academic traditions and practical experiences, is a richer environment than any locally-concentrated team could offer. I believe I will grow faster here than anywhere else.

How should Canonical amplify its impact in open source?

I do not have much idea on this but canonical can amplify impact by making it easier for external contributors to participate in its projects.

How should Canonical set about winning, commercially?

I think Canonical's strongest commercial lever is trust. Organisations that run Ubuntu in production already trust Canonical. So the opportunity is to deepen that relationship.

Outline your thoughts on the mission of Canonical. What is most appealing? What do you see as risky or unappealing?

Canonical's mission to make open source software accessible, reliable, and enterprise ready is one I find genuinely compelling. What appeals to me most is that the work is real infrastructure software that millions of people and organisations depend on.